
Algorithm 12: Bi-directional Stitcher

Input:

- V . Set of nodes
- Labels_F . Map of Pareto-optimal forward labels from Algorithm 10
- Labels_B . Map of Pareto-optimal backward labels from Algorithm 11
- Record . The global static record to beat (e.g., 13)
- MinWait . Minimum connection time required at a station (e.g., 5 or 10 mins)

Output:

- P_{best} . The reconstructed sequence of labels representing the best route, or null if no route beats the record.

```
1: best_route  $\leftarrow$  null
2: best_countries  $\leftarrow \emptyset$ 
3: best_duration  $\leftarrow \infty$ 
4: // 1. Scan the intersection of nodes
5: for all  $v \in V$  do
6:   if  $\text{Labels}_F(v) = \emptyset \vee \text{Labels}_B(v) = \emptyset$  then
7:     continue // Node was not reached by both halves
8:   // 2. Iterate through all combinations of forward and backward labels at node 'v'
9:   for all  $L_f \in \text{Labels}_F(v)$  do
10:    for all  $L_b \in \text{Labels}_B(v)$  do
11:      // — CHECK A: TIME CONTINUITY —
12:      // Does the forward path arrive in time to catch the backward path's departure?
13:      wait_time  $\leftarrow L_b.t - L_f.t$ 
14:      if wait_time < MinWait then
15:        continue
16:      // — CHECK B: 24-HOUR CONSTRAINT —
17:      // Is the total combined elapsed time strictly within 24 hours?
18:      total_duration  $\leftarrow L_b.\text{end\_time} - L_f.\text{start\_time}$ 
19:      if total_duration > 1440 then
20:        continue
21:      // — CHECK C: GEOGRAPHIC UNION —
22:      // Merge the two sets of countries (this naturally deduplicates visited countries)
23:       $C_{\text{combined}} \leftarrow L_f.C \cup L_b.C$ 
24:      // — EVALUATE AND UPDATE —
25:      // We strictly want to beat the record. If it ties the current best, we can tie-break by duration.
26:      if  $|C_{\text{combined}}| > \text{Record} \vee (|C_{\text{combined}}| = \text{Record} \wedge \text{total\_duration} < \text{best\_duration})$  then
27:        Record  $\leftarrow |C_{\text{combined}}|$  // Dynamically raise the bar for the rest of the loop
28:        best_countries  $\leftarrow C_{\text{combined}}$ 
29:        best_duration  $\leftarrow \text{total\_duration}$ 
30:        // 3. Reconstruct the stitched path
31:        best_route  $\leftarrow \text{ReconstructStitchedPath}(L_f, L_b)$ 
32: return best_route
33: // Auxiliary Function: ReconstructStitchedPath( $L_f, L_b$ )

34:  $P_{\text{forward}} \leftarrow$  empty sequence
35: curr  $\leftarrow L_f$ 
36: // Trace forward path backwards to the start
```

```

37: while curr  $\neq$  null do
38:   prepend curr to  $P_{\text{forward}}$ 
39:   curr  $\leftarrow$  curr.pred
40:  $P_{\text{backward}} \leftarrow$  empty sequence
41: // Note: We skip the first node of  $L_b$  to avoid duplicating the meeting station 'v'
42: curr  $\leftarrow L_b.\text{succ}$ 
43: // Trace backward path forwards to the destination
44: while curr  $\neq$  null do
45:   append curr to  $P_{\text{backward}}$ 
46:   curr  $\leftarrow$  curr.succ
47: return  $P_{\text{forward}}$  concatenated with  $P_{\text{backward}}$ 

```

Running Algorithm 9 and debugging, we saw that the rollouts weren't really helping, mainly because when reaching the night, the graph was much more sparse and the rollouts were not able to find good solutions. This is why we decided to split the algorithm in two, one running forward in time until half of the day, and one running backward in time until half of the day. This way we can ensure that we have a good coverage of the graph and we can find good solutions.

Both algorithms run 2 hours further than the middle of the day, giving us a 4 hour overlap to avoid missing some critical edges and hence a possible solution. The idea is to run in parallel the "same" algorithm for the two time frames, which should be faster since we run them at the same time and each of them is quite shorter, and then use the algorithm defined in this file (Algorithm 12) as a stitcher to find the best solution.

Storing the solutions in Algorithms 10 and 11 as a dictionary of routes ending at each specific node allows us to quickly find the best solution by simply iterating through the overlapping nodes and checking the best combination of forward and backward routes, making the stitching faster.